A Sharp Real-Time Pushdown Realizability Dichotomy for Linear Modular Height Guards

Alp Eren Bütün
Independent Researcher, Türkiye
ORCID: 0009-0001-2479-6037 | GitHub: alperenbutun

October 2026

Abstract

We consider two independently tagged Dyck-prefix stacks on disjoint input alphabets. A word is accepted if both stacks are correctly matched and empty at termination, and each intermediate pair of stack heights belongs to a prescribed region. For any modulus $p \geq 2$ and coefficients $a, b \in \mathbb{Z}/p\mathbb{Z}$, impose the guard that either stack is empty or $ax + by \not\equiv 0 \pmod{p}$. We establish an exact dichotomy: the resulting language has a deterministic, source-faithful, history-independent one-stack realization with one input-consuming transition per letter precisely when $a = 0$, $b = 0$, or $a + b = 0$ modulo p. In every other case the language is not context-free. The positive constructions encode the full tagged source contents, not merely their heights, using canonical finite tails and paired frames; a bounded-repair packing lemma yields real-time execution. The negative proof separates an anti-diagonal three-block obstruction from a periodic detour construction yielding two crossed equalities. The criterion applies without invertibility assumptions on a or b and admits explicit obstruction certificates of length at most $2p$ for each detour period. We distinguish these source-action-sensitive statements from existing results on multistack machines, periodic counter graphs, and regular-trajectory shuffles.

Keywords: deterministic pushdown automata; Dyck languages; canonical realization; real-time automata; modular guards; context-free languages; multistack interleavings.
2020 Mathematics Subject Classification: 68Q45, 68Q70, 68Q25.

1 Introduction

The shuffle of two Dyck languages records two independent last-in–first-out sources. Even though each source is deterministic and context-free, their unrestricted interleaving need not be context-free. A natural way to restrict the product is to prescribe which *pairs of source-stack heights* may occur at every input prefix. This paper isolates a three-parameter family for which the boundary between deterministic one-stack realization and impossibility is exact.

The model matters. Input alphabets are disjoint, so each input letter operates on exactly one source stack. The height condition is an *external all-prefix guard*: it is not a separate requirement that each of the two source machines can verify in isolation. The tags in the two stacks are retained in full; height-only counter recognition is not the claimed positive result.

For a modulus $p \geq 2$ and residues a, b, consider the guard

$$S_{p,a,b} = F_0 \cup \{(x, y) \in \mathbb{N}^2_{>0} : ax + by \not\equiv 0 \pmod{p}\}, \qquad F_0 = \{(x, y) : xy = 0\}. \tag{1}$$

The main theorem gives three algebraically described realizable cases, and proves that no pushdown automaton at all recognizes the other cases. Neither coefficient needs to be invertible modulo p.

This study continues the author's earlier programme on semantic normalization and physical realization. Previous articles introduced a framework for real-time pushdown normalization [1], examined semantic uniqueness and LIFO lifts [2], and analyzed resource tradeoffs in binary-counting DPDAs [3]. Those works are conceptual predecessors; the present article treats a distinct, externally

height-guarded *two-source* language family. Every statement needed for the modular-guard result is formulated and proved here rather than imported from a preceding manuscript.

Main contributions

(i) An exact algebraic criterion, valid for every modulus and all coefficients including zero divisors, for source-canonical strict-real-time one-stack realization of a modularly guarded pair of tagged Dyck sources.

(ii) Explicit full-source encodings using bounded tails and paired frames, together with a deterministic bounded-repair-to-real-time compilation argument.

(iii) Two complementary non-context-freeness proofs: a three-equal-block transfer obstruction and king-isolated periodic detours with explicit period-word lengths at most $2p$.

(iv) An exact enumeration of realizable coefficient pairs and a counterexample showing why height-dependent guards cannot in general be specified by a scheduling language alone.

These contributions concern the stated source/action contract, not arbitrary semilinear guards or unrestricted multi-stack realizability. Classical buffering and crossed-equality techniques are used with appropriate attribution.

The four contributions are linked by a common exact interface: a positive certificate is a full-source code with deterministic real-time update, whereas a negative certificate proves that no context-free recognizer exists at all. This distinction rules out mistaking a failure of one proposed compiler for an absolute impossibility theorem.

The paper's viewpoint differs from several established frameworks. Classical semilinear geometry and bounded-language methods are essential background [4]; periodic digraphs describe hierarchies of counter languages [6]; scope- and balance-bounded multi-stack restrictions supply other routes to effective analysis and sequentialization [8, 7]; and regular-trajectory shuffles classify restrictions imposed on the *scheduler word* [9]. Our guard depends instead on the changing source heights. The 2026 preprint [10] discusses finite buffers and crossed dependencies in CFL intersections. The cited regular-trajectory results control a projected scheduler sequence, not arbitrary constraints on the actual two source-stack heights. In fact Proposition 6.2 exhibits two words with the same scheduler and valid separate Dyck projections but different guarded membership. This proves a precise nonreduction to scheduler-only restrictions, not an unconditional novelty claim against all scheduling-related methods. These connections motivate close comparison; no claim is made that the buffering or crossed-equality arguments themselves are new.

2 Model and main theorem

Let T_X, T_Y be arbitrary nonempty finite tag sets. For $t \in T_X$ write x_t for a push on X and $\bar{x}_t$ for its matching pop. Let $y_s, \bar{y}_s$ be the corresponding Y letters. The alphabets are disjoint. Each source stack is initially empty and a pop is enabled only when its tag agrees with the current top. Let $\mathrm{ht}_X(w), \mathrm{ht}_Y(w)$ be the source heights after a legal prefix w. A word is *guarded legal* when every prefix, including the empty prefix, has both source operations defined and satisfies (1). Let $L_{p,a,b}$ consist of the guarded legal words whose two final source stacks are empty.

Definition 2.1 (Exact rooted source-canonical real-time realization)**.** A one-stack implementation is *source faithful* if its reachable configurations at consumed-input boundaries admit a decoder to the complete ordered pair of tagged source stacks, and the decoder both reflects transition definedness and intertwines each input action with the corresponding source action. It is *rooted canonical* when, on reachable source configurations, there is a unique encoding $\sigma(U, V)$, independent of the input history, with the empty source pair encoded at the physical root. It is *strict real-time* when no ε-transition is used and each input symbol is processed in one pushdown transition with at most one physical top-stack edit. The state and physical alphabet may depend on the fixed parameters p, a, b, T_X, T_Y.

Theorem 2.2 (Sharp modular-guard dichotomy)**.** *For every $p \geq 2$ and $a, b \in \mathbb{Z}/p\mathbb{Z}$ the following are equivalent:*

(i) *$a = 0$, $b = 0$, or $a + b = 0$ in $\mathbb{Z}/p\mathbb{Z}$;*

(ii) *$L_{p,a,b}$ admits an exact rooted source-canonical strict-real-time one-stack implementation;*

(iii) *$L_{p,a,b}$ is context-free.*

If the equivalent conditions fail, $L_{p,a,b}$ is not context-free; the obstruction excludes even nondeterministic PDAs. All congruences in (i) are tested separately, including when p is composite.

We prove the constructive direction in Sections 3–4 and the negative direction in Section 5. In the nonzero diagonal case the machinery works with the period $q = p/\gcd(p, a)$; consequently nonunits pose no exceptional case.

3 Canonical positive realizations

We first state the two normal forms underlying the positive half. A stack frame may store, in addition to its tag(s), any fixed finite summary such as the previous height modulo q and whether a frame is the first of its kind. These annotations are uniquely determined by the source contents and the chosen normal form.

3.1 Axis and vertical-stripe cases

If $a = b = 0$, the positive quadrant is forbidden. At most one source stack is nonempty; the physical stack stores that stack verbatim, with a finite control specifying the active source. Every attempted transition to two nonempty sources is rejected. This is immediately real-time and canonical.

Suppose $b = 0$, $a \neq 0$, and put $q = p/\gcd(p, a) \geq 2$. In the positive quadrant,

$$ax \not\equiv 0 \pmod{p} \quad \Longleftrightarrow \quad x \not\equiv 0 \pmod{q}. \tag{2}$$

Define $H_q = \{0\} \cup \{x \geq 1 : x \not\equiv 0 \pmod{q}\}$. Its nearest-neighbor components are

$$[0, q-1],\ [q+1, 2q-1],\ [2q+1, 3q-1], \ldots. \tag{3}$$

For $x \in H_q$ let $\lambda(x)$ denote the left endpoint of the component containing x, and let $d(x) = x - \lambda(x)$. Always $0 \leq d(x) \leq q - 1$.

Lemma 3.1 (Canonical stripe normal form)**.** *For the guard $F_0 \cup \{(x, y) \in \mathbb{N}^2_{>0} : x \not\equiv 0 \pmod{q}\}$, the tagged source pair admits a history-independent one-stack normal form. Every input step, including normalization, is realizable by a deterministic sequence of at most $q + O(1)$ physical top operations. All decisions depend only on finite control and the physical top frame.*

Proof. When $y = 0$, the physical stack is the complete tagged X stack; Y is empty. When $y > 0$, the physical stack, bottom to top, consists of the first $\lambda(x)$ tagged X frames followed by all Y frames except its bottommost one. The finite controller retains the last $d(x)$ tags of X and the bottommost tag of Y. If $x = 0$ set $\lambda(0) = d(0) = 0$. These data specify a unique, injective encoding of (X, Y).

A transition $y = 0 \to 1$ is allowed only if $x \in H_q$. It removes the last $d(x) \leq q - 1$ physical X frames into the finite tail and caches the new Y tag. The reverse transition $y = 1 \to 0$ reattaches exactly the bounded tail after verifying the last Y tag. For $y > 0$, every legal change of x remains within the same connected component of H_q: any move to another component would cross a forbidden height divisible by q. Hence $\lambda(x)$ is fixed and only the bounded finite X tail changes. A purported X pop below the left endpoint is rejected from the known height/residue boundary, before examining any buried frame. In this mode Y pushes and pops use physical frames above the frozen X base, retaining the bottom Y tag as a boundary marker in the controller. When $y > 1 \to 1$, a distinguished first excess frame records the crossover. The controller knows whether the physical top is a Y frame and its tag, or the exposed base.

Modulo-q counters and finite predecessor summaries stored on the frames suffice to check all height conditions without inspecting the buried stack. Each case produces exactly the specified

normal form for the updated sources; rejected actions leave it unchanged. The only multi-operation conversions move at most $q-1$ X frames. Induction from the empty source pair proves exact domain reflection, decoding, and path-independent canonicality. □

Exact boundary transitions. For clarity, let t_X denote the finite X-tail and let Y_0 denote the cached bottom Y tag in the $y > 0$ mode. Every consumed symbol falls into one of the following mutually exclusive cases (the reverse, disabled operations have their usual typed-top checks):

Source condition	Quiescent update before real-time packing
$y = 0$, X operation	Update the physical X stack and the height summary; reject an illegal typed pop.
$y = 0$, Y push	If the new interior height is allowed, transfer the last $d(x)$ X tags to t_X and store the new Y_0 in control; otherwise reject without mutation.
$y = 1$, Y pop	Verify Y_0, restore t_X to the X stack, clear the Y mode.
$y \geq 1$, X operation	Check the new guard and source-top tag using the finite tail and its component boundary. Update only t_X; no crossing to another H_q component is legal.
$y \geq 1$, Y operation, not at $1 \to 0$	Push/pop the top excess-Y frame, or change between $y = 1$ and $y = 2$ using its distinguished boundary frame.

The components in (3) are separated by a forbidden positive X height; hence no legal interior update of X requires access to the buried X base. This is the point at which a global height restriction becomes a bounded top-of-stack operation.

If $a = 0$, $b \neq 0$, the same lemma applies after exchanging the X and Y roles.

3.2 Diagonal-stripe case

Suppose $a + b = 0$ and $a \neq 0$. Write $b = -a$ and $q = p/\gcd(p, a) \geq 2$. Then

$$ax + by \not\equiv 0 \pmod{p} \quad \Longleftrightarrow \quad x - y \not\equiv 0 \pmod{q}. \tag{4}$$

When $x, y > 0$, this implies $x \neq y$. Put $d = |x - y| = kq + r$ with $k \geq 0$ and $1 \leq r \leq q - 1$.

Lemma 3.2 (Canonical paired-frame normal form)**.** *Under the guard $F_0 \cup \{x, y > 0 : x - y \not\equiv 0 \pmod{q}\}$, there exists a unique, history-independent one-stack normal form preserving all tags. Each source action updates it using at most $q + O(1)$ deterministic top operations.*

Proof. At the origin the physical stack is empty. On either axis it stores exactly the nonempty source stack as single-source frames. In the positive quadrant, assume $x > y$; the other case is symmetric. The physical stack consists, bottom to top, of the first kq tagged X symbols as single frames, followed by y paired frames. The ith pair stores the $(kq + i)$th X tag and the ith Y tag, both indexed from the bottom. The remaining r top X tags form a finite-control tail. A marker on the lowest paired frame permits the controller to recognize when the last pair is removed. This decomposition is unique for the source pair.

In the interior, a majority-side push/pop changes r by ± 1. A minority push takes the lowest tag from the finite majority tail and puts it together with the new minority tag into a top paired frame, reducing r by one. A minority pop removes the top pair and places its majority tag at the beginning of the finite tail, increasing r by one. Whenever this would give a forbidden difference divisible by q, the action is rejected *before* altering the representation. The block of kq majority tags cannot change while the run stays in this interior band.

For a minority pop $1 \to 0$, the entire pair/tail interface is converted into ordinary majority frames. The number of frames restored is at most $r + 1 \leq q$. Conversely, moving from an axis into the interior consumes at most q top majority tags to form the first pair and its finite tail. These are deterministic top-only conversions; the chosen normal form is restored at every input boundary. The residue class, mode, and first-pair marker are finite data. All actions, source-top tag tests, and guard checks are therefore exact and implementable with a constant number of microsteps depending on q. □

Reversible pairing at the band boundary. For $x > y > 0$, the paired-frame count is precisely y and the control tail has length r. The four interior updates can be checked algebraically: a majority push/pop changes d by ± 1 and extends/shortens the tail; a minority push/pop changes d by ∓ 1 and transfers one majority tag between the beginning of the tail and the top pair. At $r = 1$ a minority push is rejected unless it is an axis operation, since it would create a positive-positive difference divisible by q; at $r = q - 1$ a minority pop is rejected unless it empties the minority source. These exceptions are exactly the axis crossover cases already treated in the lemma. In particular no allowed interior operation moves the kq-frame base or changes the identity of its boundary. All stored finite summaries may be reconstructed deterministically from the final tagged source pair, so the encoding is not a trace-dependent normal form.

4 From bounded repairs to a strict real-time PDA

The preceding two lemmas give deterministic quiescent one-stack encodings with bounded intermediate normalization. For clarity we spell out the (classical-style) stack-blocking step required to obtain one consumed-input transition per source letter; we do not claim this acceleration technique as an original independent result; compare the classical real-time strict-deterministic equivalence in [5], whose hypotheses and conclusions are not being silently substituted for the explicit construction below.

Lemma 4.1 (Block-packing acceleration)**.** *Suppose a deterministic one-stack implementation over a fixed finite alphabet has a unique quiescent configuration at each input boundary, and each input letter induces a deterministic sequence of at most C stack-top microsteps from one quiescent configuration to another (or detects rejection), where C does not depend on input length or stack depth. Then it admits a deterministic ε-free real-time simulation, preserving its quiescent source decoder and history-independent encoding if the original encoding had these properties.*

Proof. Choose $B \geq C + 2$. Partition the simulated stack into a physical stack of full blocks of B old symbols and a top word cached in finite control. Normalize the packing as follows: if the old stack has fewer than B symbols, cache all of it; otherwise cache the final $B + ((n - B) \bmod B)$ symbols of a stack of length n and partition the lower prefix into blocks of exactly B. Thus, whenever a hidden block exists, the cache has length between B and $2B - 1$.

On reading one input symbol, execute the complete bounded microprogram in the finite controller using the cached top word, and inspect the (single) visible top block if needed. The microprogram can remove at most $C < B$ old symbols, so it never needs a symbol below the starting cache. Afterward the cache has changed in length by at most C. Restore normalized packing by at most one physical block operation: push the bottom B cached symbols if the cache is too long, pop the top hidden block and prepend its symbols if the cache is too short, or do nothing. Because $B \geq C + 2$, one correction is enough. Before committing a physical push/pop, the control determines the entire bounded microprogram on a temporary copy of its finite cache. If that program would reject, the compiled transition is undefined and no physical state is changed. If it succeeds, at most one packed-stack block is pushed or popped. The block is a *single symbol* of the new finite stack alphabet, so this is one permitted PDA edit, not B edits. The initial packing is canonical because its block count and cache length are uniquely determined by n; the post-step packing is uniquely normalized by the same rule. All possible cached words, microprogram branches, and full-block symbols range over finite sets, yielding a finite deterministic real-time transition table. The packing uniquely encodes the prior quiescent stack and preserves the full decoder and canonical section by composition. □

Proposition 4.2 (Positive half of Theorem 2.2)**.** *If $a = 0$ or $b = 0$ or $a + b = 0$ modulo p, then $L_{p,a,b}$ has an exact rooted source-canonical strict-real-time one-stack implementation.*

Proof. The zero-zero case uses the axis construction. In the one-zero cases use (2), Lemma 3.1, and symmetry. In the remaining case use (4) and Lemma 3.2. Lemma 4.1 converts each bounded-repair

implementation to strict real time while maintaining unique source encoding and exact definedness. Both tagged sources are empty exactly in the root normal form, which is the acceptance condition. □

5 Two independent impossibility mechanisms

We write $x, \bar{x}, y, \bar{y}$ for a fixed single tag of each source, so all negative witnesses use a regular subalphabet of any richer tag set. For a signed source word w, let w^{-1} denote reversal with all pushes and matching pops exchanged, so its signed-height path is traversed backward.

5.1 Anti-diagonal transfer: three equal blocks

Lemma 5.1 (Three-block obstruction)**.** *If $a, b, a+b$ are nonzero modulo p and $a = b$ modulo p, then $L_{p,a,b}$ is not context-free.*

Proof. Let $q = p/\gcd(p, a)$. Since $a \neq 0$ and $2a = a + b \neq 0 \pmod{p}$, one has $q \geq 3$. The forbidden positive-quadrant points are exactly those satisfying $x + y \equiv 0 \pmod{q}$. For each $n \geq 0$, set $N = qn + 2$ and consider

$$x^N (\bar{x}y)^N \bar{y}^N. \tag{5}$$

The first and last blocks lie on axes; during the middle block the sum of heights alternates between $N - 1$ and N, which are 1 and 2 modulo q. Thus every word in (5) is guarded legal and ends with both stacks empty.

Intersect $L_{p,a,b}$ with the regular pattern $x^2(x^q)^*(\bar{x}y)^*\bar{y}^*$. After treating each $\bar{x}y$ as a single letter by inverse homomorphism, the resulting language is

$$\{x^N z^N \bar{y}^N : N \equiv 2 \pmod{q},\ N \geq 2\}.$$

Indeed, the final emptiness of X and Y forces all three exponents to coincide, while the preceding prefix check verifies all equal-count words. This language is not context-free by the standard three-equal-block pumping argument, which is unchanged by restricting N to one infinite arithmetic progression. Closure under regular intersection and inverse homomorphism gives the claim; applying the inverse map to the three-block macro alphabet is legitimate because each transfer factor $\bar{x}y$ is a fixed word. □

5.2 Isolated holes: periodic detours

Lemma 5.2 (Detour certificates)**.** *Suppose $a, b, a+b, a-b$ are all nonzero modulo p. There are two explicitly constructible source words P, Q, each of length at most $2p$, with signed height displacements $(p, 0)$ and $(0, p)$, such that their paths are guard legal from $(1, 1)$ and all translations of these paths by (ip, jp) for $i, j \geq 0$ are guard legal. Consequently $L_{p,a,b}$ is not context-free.*

Proof. A forbidden positive-quadrant point has $ax + by \equiv 0 \pmod{p}$. Two such points cannot be eight-neighbors, since their displacement would give one of $\pm a$, $\pm b$, $\pm(a + b)$, or $\pm(a - b)$ equal to zero modulo p. Thus deleted cells are isolated even under diagonal adjacency.

The point $(1, 1)$ is permitted because $a + b \neq 0$. Starting there, traverse the row $y = 1$ from $x = 1$ to $x = p + 1$. If the next cell $(h, 1)$ is forbidden, replace the two horizontal moves through it by the four-step bypass $yxx\bar{y}$, traveling from $(h - 1, 1)$ to $(h + 1, 1)$. Isolation guarantees that the bypass's intermediate cells, including those on the row $y = 2$, are permitted. All deleted cells are nonadjacent, so bypasses do not conflict. The resulting word P has displacement $(p, 0)$. Construct Q symmetrically along $x = 1$, replacing two steps through a forbidden $(1, h)$ by $xyy\bar{x}$; its displacement is $(0, p)$. The endpoints $(p + 1, 1)$ and $(1, p + 1)$ are permitted by periodicity.

If h bypasses occur on a route of p elementary axial moves, the length is $p + 2h$. There are at most $\lfloor p/2 \rfloor$ bypasses, so $|P|, |Q| \leq 2p$. The membership congruence and the positive-quadrant condition are invariant under translations by (ip, jp), $i, j \geq 0$, establishing the claimed path property.

Let $u = xy$ be the legal entry to $(1, 1)$. For all $m, n \geq 0$, the word

$$uP^m Q^n (P^{-1})^m (Q^{-1})^n u^{-1} \tag{6}$$

is guarded legal and empties both sources, by periodicity and reversal of the appropriate translated paths. For arbitrary nonnegative exponents m, n, k, ℓ, replace the two inverse-block exponents in (6) by k, ℓ. Final source emptiness forces $p(m - k) = p(n - \ell) = 0$, hence $m = k$ and $n = \ell$. Conversely balanced exponents give (6) and are accepted.

Using four fresh macro letters and the homomorphism mapping them to P, Q, P^{-1}, Q^{-1}, followed by fixed left/right quotients with u, u^{-1} and intersection with the regular four-block pattern, one obtains exactly

$$\{\alpha^m \beta^n \gamma^m \delta^n : m, n \geq 0\},$$

which is not context-free. Hence $L_{p,a,b}$ cannot be context-free. □

Proof of Theorem 2.2. Proposition 4.2 establishes (i)⇒(ii); (ii)⇒(iii) is immediate. If (i) fails, then $a, b, a + b$ are all nonzero. When $a - b = 0$, Lemma 5.1 excludes context-freeness; when $a - b \neq 0$, Lemma 5.2 excludes it. This exhaustive split proves (iii)⇒(i). □

6 Consequences and action-sensitive scope

Corollary 6.1 (Coefficient census)**.** *For any $p \geq 2$, among the p^2 ordered coefficient pairs (a, b) there are exactly $3p - 2$ realizable pairs and $(p - 1)(p - 2)$ non-context-free pairs. In particular the realizable proportion is $3/p - 2/p^2 \to 0$.*

Proof. The three length-p lines $a = 0$, $b = 0$, and $a + b = 0$ intersect pairwise and jointly only at $(0, 0)$. Inclusion–exclusion gives $3p - 3 + 1 = 3p - 2$. Subtracting from p^2 yields the second count. □

Proposition 6.2 (A schedule word does not determine the guard)**.** *There are two correctly tagged, independently balanced source words with the same X/Y scheduling word such that one belongs to a modularly guarded language of the above family and the other does not. Thus, even for a fixed pair of Dyck source languages, the guard cannot in general be represented solely by a scheduling-language restriction, whether regular or otherwise.*

Proof. Take $p = 3$ and $a = b = 1$ and use one tag per source. The two words

$$w_1 = x\bar{x}yx\bar{x}\bar{y}, \qquad w_2 = xxy\bar{x}\bar{x}\bar{y}$$

have the common scheduling sequence $XXYXXY$. Both projections are Dyck words. The first never meets a forbidden positive-positive sum divisible by three; the second passes through $(2, 1)$ and fails the guard. Any membership restriction depending only on the scheduling word would treat both alike. □

It is essential not to silently allow one letter to alter both source stacks. Such simultaneous actions change the accessible height graph and can change context-freeness even under the same geometric guard. Likewise the theorem is not a classifier for all Presburger height guards or all two-source control systems. For a source-preserving implementation, the decoder and canonical section satisfy the usual rooted intertwining and domain-reflection equations; such an instance does not by itself establish a new universal realization axiom or a quantitative assembly-deficit formula.

A finite arithmetic decision and a constructive negative witness. The input p, a, b is first reduced modulo p. Three modular equality tests decide the positive branch, including composite moduli; no multiplication-based zero-divisor test is sound here. In the negative branch the test $a = b$ chooses the three-block witness; otherwise the two detour words are constructed by inspecting at most p consecutive row/column positions, each with constant-length replacements. The algebraic decision is polynomial in the binary size of the coefficients, whereas explicitly writing these periodic witnesses can require length proportional to p. The positive device is effective for each fixed parameter triple but its alphabet and control size need not be bounded polynomially in $\log p$.

Finite checks. Independent scripts checked parameter instances (including composite moduli), detour paths, and representative tagged-source transitions. These checks are supplementary consistency tests; the results above rest on their symbolic proofs. The explicit detour words have length $O(p)$, which is not polynomial in the binary input length $\log p$.

Statements and Declarations

Funding. No external funding is declared for this study.
Competing interests. The author declares no competing interests.
Data availability. This is a theoretical study; no external dataset was used.
Code availability. Independent finite audit programs and summary results were used for internal verification. The reproducibility archive accompanying the manuscript contains the finite audit scripts and summaries; these checks are ancillary to the symbolic proof.
Author contributions. The sole author is responsible for the mathematical content and manuscript.
Ethics approval and consent. Not applicable.

References

[1] A. E. Bütün, "A General Theory of Real-Time Pushdown Normalization: Semantic Quotients, Locality, Representation Debt, and Causal Scheduling," Zenodo preprint (2026). doi:10.5281/zenodo.22709302.

[2] A. E. Bütün, "Semantic Uniqueness and LIFO Lift Theory for Real-Time Pushdown Normalization," Zenodo preprint (2026). doi:10.5281/zenodo.22709334.

[3] A. E. Bütün, "Exact Structure and Resource Bounds in Deterministic Pushdown Automata for Binary Counting Languages," Zenodo preprint (2026). doi:10.5281/zenodo.22709367.

[4] S. Ginsburg and E. H. Spanier, "Semigroups, Presburger formulas, and languages," *Pacific Journal of Mathematics* **16** (1966), 285–296. doi:10.2140/pjm.1966.16.285.

[5] M. A. Harrison and I. M. Havel, "Real-time strict deterministic languages," *SIAM Journal on Computing* **1** (1972), 333–349. doi:10.1137/0201024.

[6] N. Jonoska, M. Krajčevski, and G. McColm, "Counter machines and crystallographic structures," *Natural Computing* **15** (2016), 97–113. doi:10.1007/s11047-015-9527-0.

[7] H. J. S. Basten, "Context-free ambiguity detection using multi-stack pushdown automata," in *Developments in Language Theory, DLT 2016*, LNCS 9840, Springer, 2016, pp. 1–12. doi:10.1007/978-3-662-53132-7_1.

[8] S. La Torre, M. Napoli, and G. Parlato, "Scope-bounded pushdown languages," *International Journal of Foundations of Computer Science* **27** (2016), 215–233. doi:10.1142/S0129054116400074.

[9] C. Barloy, M. Cadilhac, and K. Ockerlund, "Shuffles of context-free languages along regular trajectories," *ICALP 2026*, LIPIcs **374** (2026), 163:1–163:17. doi:10.4230/LIPIcs.ICALP.2026.163.

[10] J. M. Silva, "Geometric characterization of context-free intersections via the inner segment dichotomy," preprint (2026), arXiv:2602.14722.